

DAGSmith: Dependency-Aware Rewriting for dbt-Style SQL Pipelines

Jie Liu

jiezzliu@umich.edu

Computer Science and Engineering
University of Michigan, Ann Arbor
USA

Lin Ma

linmacse@umich.edu

Computer Science and Engineering
University of Michigan, Ann Arbor
USA

Barzan Mozafari

mozafari@umich.edu

Computer Science and Engineering
University of Michigan, Ann Arbor
USA

ABSTRACT

Modern analytics is increasingly organized as recurring SQL pipelines rather than isolated SQL statements. Tools such as dbt—which have gained extreme popularity in recent years—allow teams to write each transformation as SQL and make dependencies between transformations explicit, producing directed acyclic graphs (DAGs) with hundreds or thousands of interdependent SQL models. Existing techniques are not effective at optimizing these expensive pipelines. Traditional query optimizers and source-to-source query rewriters operate on one query at a time, while materialized-view selection and multi-query optimization address narrower forms of reuse. They do not directly exploit the pipeline-level information exposed by explicit dependencies: how intermediate results are consumed, which downstream outputs depend on each computation, where expensive work occurs relative to data reduction, which intermediate results are worth persisting, and how refresh schedules relate to the rate at which inputs change and outputs are consumed.

We introduce DAGSmith, to the best of our knowledge the first holistic dependency-aware source-to-source rewriting system for SQL pipeline DAGs. DAGSmith treats explicit dependencies as optimization signals. It analyzes each transformation together with its upstream inputs, downstream consumers, and position in the pipeline DAG, uses an LLM to propose pipeline-level refactorings, separates SQL generation and **data-backed equivalence checks** to reject unsafe rewrites, retunes persistence choices with a learned cost model, and selects a globally compatible set of rewrites that avoids conflicts across transformations. This enables dependency-edge simplification, non-local semantic reuse, downstream-aware pruning, pipeline-aware work placement, rewrite-materialization co-optimization, and frequency-aware optimization while keeping the pipeline executable by the same SQL engine and orchestration framework. Our extensive evaluation shows that, on the open-source Tuva dbt project, DAGSmith reduces elapsed time by **42.6%** and warehouse compute cost by **67.7%**; these reductions are **22.8%/83.0%** larger than materialization-only tuning and **98.1%/348.3%** larger than state-of-the-art single-query rewriting techniques.

1 INTRODUCTION

Rising SQL Complexity — SQL is no longer only a hand-written interface for individual analytical questions. It is increasingly the implementation language for recurring data products: governed dashboards, AI-ready feature tables, self-service BI datasets, and organization-wide reporting layers. At the same time, more SQL is produced by BI systems, code generators, and AI-assisted development tools rather than written from scratch. In fact, recent

studies report that 70% of analytics professionals use AI for code development [9], and that BI-generated SQL contain extremely complex scalar expressions and relational operator trees [28]. This has created a major efficiency gap between the complexity of the SQL queries generated and what today’s query optimizers are capable of effectively optimizing. Source-to-source query rewriting is the traditional way to narrow this gap: a rewriter transforms complex SQL into an equivalent form that the query optimizer is more likely to turn into an efficient plan. However, query rewriting has traditionally treated one SQL statement as the unit of optimization. Unfortunately, the effectiveness of this one-query-at-a-time approach is increasingly limited by a second trend: modern SQL is no longer produced and executed only as isolated statements, but as recurring transformation pipelines.

Rise of SQL Pipelines — Data teams are increasingly building large, recurring SQL pipelines. This growth is driven by complex business logic that must be computed repeatedly and reused consistently across governed dashboards, AI-ready feature tables, self-service BI datasets, compliance reports, and downstream applications. One of the most widely adopted tools for these workflows is *dbt* (the data build tool) [12], reported to be used by 50K+ data teams weekly across the world [8]. *dbt* lets teams write transformations as version-controlled SQL SELECT statements (called “models” in *dbt*, which we use interchangeably in this paper), declare dependencies among them, and compile the resulting graph into SQL jobs executed by a data warehouse. Figure 1 gives a toy example. A raw orders table feeds a cleaning transformation; the cleaned result feeds both a revenue table and a daily active-customer table. The SQL text explains each local transformation, while the edges explain how the transformations compose into a pipeline. This trend is much broader than *dbt*. SQLMesh [26], Dagster asset graphs [5], Airflow directed acyclic graphs [2], Prefect flows [21], Argo Workflows [3], Databricks Workflows [6], and medallion lakehouse pipelines [7] all represent recurring data transformations through explicit dependencies. In this paper, we target the SQL subset of this broader pattern, which we refer to as *SQL pipeline DAG*: a recurring directed acyclic graph whose nodes are SQL transformations or SQL-produced tables/views, and whose edges record that one node reads another node’s output.

Pipeline Optimization Gap — These pipeline DAGs often include hundreds or thousands of SQL transformations that reference one another. Their scale makes them difficult to optimize with techniques designed for a single SQL statement or a flat workload: inlining every referenced transformation into each downstream query would produce deeply nested SQL with tens or hundreds of layers whose size and complexity are well beyond what today’s

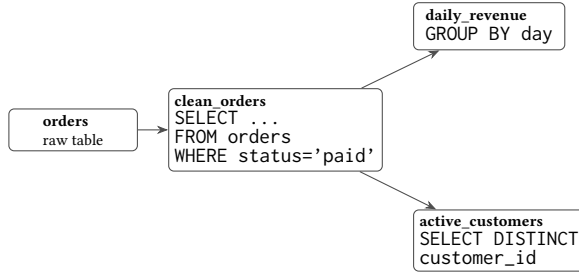


Figure 1: A toy SQL pipeline DAG. Nodes are SQL transformations or SQL-produced tables (called “models” in dbt); edges show which transformation consumes another transformation’s output.

optimizers can even tackle. In particular, existing *query rewriting* techniques, whether traditional rule-based systems [30] or modern synthesis- or LLM-based techniques [10, 17, 25], rewrite one SQL statement at a time. This scope is useful for improving a standalone query, but it is too narrow for optimizations whose correctness or benefit depends on the surrounding pipeline, such as removing work no final output uses, reusing equivalent logic across separate transformations, or changing refresh frequency based on input changes and output consumption. *Multi-query optimization* techniques [22, 23] consider multiple queries together, but their classical goal is to share common scans, joins, or intermediate results across a batch of submitted queries. They do not generally rewrite a SQL pipeline by adding, removing, splitting, or merging transformations based on downstream demand and refresh behavior. *Materialized-view selection* [1, 14] chooses intermediate query results to precompute and rewrites future queries to read those precomputed results. This improves reuse when the relevant computation has already been identified, but it does not by itself determine that a pipeline should drop unused work, factor repeated logic written in different forms, move expensive operations after data reduction, or refresh different parts of the pipeline at different rates. Lastly, *Workflow schedulers* [2, 3, 5, 21] know dependencies, but usually optimize execution order, failure recovery, retries, or resource allocation rather than the SQL semantics of the transformations themselves.

Dependencies as Signals — Our key insight is that explicit dependencies are not merely execution constraints; they are signals that can be leveraged for optimization. In particular, a SQL pipeline DAG exposes six kinds of information that are hidden or much harder to recover when queries are optimized independently: **(1) Edge semantics:** producer-consumer edges reveal how one SQL transformation’s output is used by later transformations; **(2) Cross-graph redundancy:** similar business logic can appear in distant parts of the graph, even when SQL text differs; **(3) Downstream demand:** downstream dependencies reveal which columns, joins, filters, and intermediate results can affect final outputs; **(4) Operator placement:** the graph reveals where expensive operations occur relative to data-reducing filters, projections, joins, and aggregations; **(5) Rewrite-persistence coupling:** unlike single-query rewriting, which is constrained to equivalent rewrites that are cheaper in isolation, pipeline rewriting can consider alternatives that produce additional reusable outputs and are therefore locally more expensive, as long as those outputs are materialized once and reused by

enough downstream consumers; and **(6) Refresh patterns:** schedules and source update rates reveal how often inputs change and outputs are consumed.

Our Approach — In this paper, we introduce DAGSmith, the first (to the best of our knowledge) holistic, dependency-aware rewriting system for SQL pipeline DAGs. DAGSmith exploits the exposed dependency structure to perform six classes of optimization: **(1) dependency-edge simplification:** remove or simplify dependency edges whose purpose can be expressed more directly, while preserving final outputs; **(2) non-local semantic reuse:** factor repeated work into a shared computation across non-adjacent transformations; **(3) downstream-aware pruning:** remove work that is provably irrelevant to every final output; **(4) pipeline-aware work placement:** move expensive work after data-reducing steps as long as final outputs are not impacted; **(5) rewrite-materialization co-optimization:** decide when rewrite benefits are outweighed by materialization cost, and when materializing a rewritten result makes the rewrite worthwhile; and **(6) frequency-aware optimization:** align computation frequency with input-change rates and output-consumption rates, avoiding refreshes that do not impact consumers.

DAGSmith first analyzes compiled SQL and dependency edges to identify regions likely to contain these opportunities. It then leverages Large Language Models (LLMs) to reason over the relevant subgraph and propose concrete refactorings, but separates proposal, criticism, SQL generation, and **data-backed equivalence checking** so unsafe or counterproductive rewrites can be rejected before adoption. Adopted candidates are not evaluated in isolation: DAGSmith retunes persistence choices for each candidate using a learned cost model and an iterative local linearization procedure, then solves a global selection problem to choose a non-conflicting subset of rewrites. Finally, frequency information from source update rates and output demand reweights cost estimates, detects over-scheduled work, and generates splitting candidates when slow-changing logic is embedded in fast-refreshing transformations. The result is a rewritten, output-equivalent SQL pipeline that remains executable by the same SQL engine and orchestration framework, but is significantly more performant.

Contributions — This paper makes the following contributions:

- (1) We formulate dependency-aware source-to-source rewriting for SQL pipeline DAGs. We present the first holistic approach to this problem (to the best of our knowledge) by identifying the pipeline-level signals exposed by explicit dependencies and the rewriting opportunities they enable: dependency-edge simplification, non-local semantic reuse, downstream-aware pruning, pipeline-aware work placement, rewrite-materialization co-optimization, and frequency-aware optimization.
- (2) We present the design of DAGSmith, which combines structural graph analysis, LLM-guided refactoring, separated proposal, criticism, SQL generation, and **data-backed equivalence checking**, learned cost modeling, persistence tuning, frequency-aware scheduling, and global candidate selection.
- (3) We conduct a comprehensive evaluation showing that, on the open-source Tuva dbt project, DAGSmith reduces elapsed time by **42.6%** and warehouse compute cost by **67.7%**; these reductions are **22.8%/83.0%** larger than materialization-only tuning

Table 1: Optimization signals exposed by explicit SQL pipeline dependencies.

#	What explicit dependencies expose	How DAGSmith leverages them for optimization
1	Producer-consumer edges reveal how one SQL transformation’s output is used by later transformations.	Dependency-edge simplification: remove or simplify dependency edges whose purpose can be expressed more directly, while preserving final outputs.
2	Similar business logic can appear in distant parts of the graph, even when SQL text differs.	Non-local semantic reuse: factor repeated work into a shared computation across non-adjacent transformations.
3	Downstream dependencies reveal which columns, joins, filters, and intermediate results can affect final outputs.	Downstream-aware pruning: remove work that is provably irrelevant to every final output.
4	The graph reveals where expensive operations occur relative to data-reducing filters, projections, joins, and aggregations.	Pipeline-aware work placement: move expensive work after data-reducing steps as long as final outputs are not impacted.
5	Dependency fanout reveals when a transformation could produce additional reusable outputs for downstream consumers.	Rewrite-materialization co-optimization: consider locally more expensive rewrites that pay off only when the richer result is materialized once and reused by enough downstream consumers.
6	Schedules and source update rates reveal how often inputs change and outputs are consumed.	Frequency-aware optimization: align computation frequency with input-change rates and output-consumption rates, avoiding refreshes that do not impact consumers.

and **98.1%/348.3%** larger than state-of-the-art single-query rewriting techniques.

The rest of the paper is organized as follows. Section 2 defines the setting and illustrates the opportunity space. Section 3 presents the design of DAGSmith. Section 4 outlines the evaluation plan, Section 5 discusses prior work, and Section 6 concludes.

2 PROBLEM SETTING

Before presenting our approach, we first provide the necessary background on SQL pipeline DAGs (§2.1), then present a few motivating examples of pipeline-level optimization opportunities exposed by explicit dependencies (§2.2). Lastly, we explain why local query rewriting is insufficient for this setting (§2.3).

2.1 Background: SQL Pipeline DAGs

A *SQL pipeline DAG* is a recurring SQL program whose nodes are named transformations and whose directed edges record producer-consumer relationships among their outputs (see Figure 1 for a toy example). The DAG determines evaluation order, while node metadata can specify whether each result is materialized and how frequently it is refreshed. Because dbt calls each named SQL transformation a *model*, we use *model* for dbt-specific examples and *transformation* for the broader setting.

Target workload — Although we use dbt in our examples and evaluation, the optimization problem is more general. DAGSmith targets recurring SQL-based DAGs: pipelines in which node logic is available as SQL, edges expose producer-consumer dependencies, and system metadata records persistence and refresh behavior. dbt is a prominent instance of this setting, but similar dependency graphs arise in SQLMesh [26], Dagster [5], Airflow [2], Prefect [21], Argo [3], Databricks Workflows and Lakeflow Jobs [6], medallion lakehouse pipelines [7], and Luigi [18].

2.2 Motivating Examples

In this section, we use a small claims-processing DAG to illustrate how explicit dependencies expose optimization opportunities that are invisible when each query is rewritten in isolation.

For space, the examples combine multiple opportunities in one small claims-processing DAG; in practice, each opportunity can appear independently inside much larger DAGs with hundreds or thousands of models. Figure 2(a) shows the original pipeline. Here, we use a simplified healthcare claims-processing pipeline, inspired by the open-source Tuva project [27]. Its main input is `claims`, a large table of claim-line records that is refreshed frequently. It feeds into service-category models such as `dialysis`, `psychiatric`, `equipment`, and `nursing`. These models classify claim lines using SQL predicates over claim attributes; Figure 3 gives simplified examples. The `nursing` model excludes claim lines already categorized as `equipment`, so it reads `equipment` as an exclusion list. The `claims_summary` model joins claim lines with a slow-changing lookup table, `code_catalog`, that maps raw claim codes to descriptions, and then applies code-grouping logic over those descriptions. We next describe three examples that exploit these explicit dependencies; when all applied together, they produce the more efficient DAG in Figure 2(b).

Example 1: Cross-graph redundancy + rewrite-persistence coupling. In Figure 2(a), the fanout from `claims` to the service-category models (`dialysis`, `psychiatric`, `equipment`, and `nursing`) exposes cross-graph redundancy: several models scan the same large source (i.e., `claims`), and evaluate related billing-code, taxonomy-code, or diagnosis-code predicates in separate SQL files. By exploiting this redundancy, one can create the shared `service_flags` model in Figure 2(b) and rewrite each of the downstream models to read the corresponding flag. Figure 3a shows the rewrite for `dialysis`; the other service-category models can be rewritten similarly by reading their own flags. In §1, we call this non-local semantic reuse.

This shared model also creates a materialization choice: computing extra flags such as `f_psy` pays off only when `service_flags` is persisted once and reused, rather than inlined under each consumer. This is rewrite-materialization co-optimization in §1.

Example 2: Edge semantics + downstream demand. We can also exploit the edge semantics in Figure 2(a): the edge from `equipment` to `nursing` implies that `nursing` consumes `equipment`, but the SQL definitions show that `nursing` uses `equipment` only to exclude claim

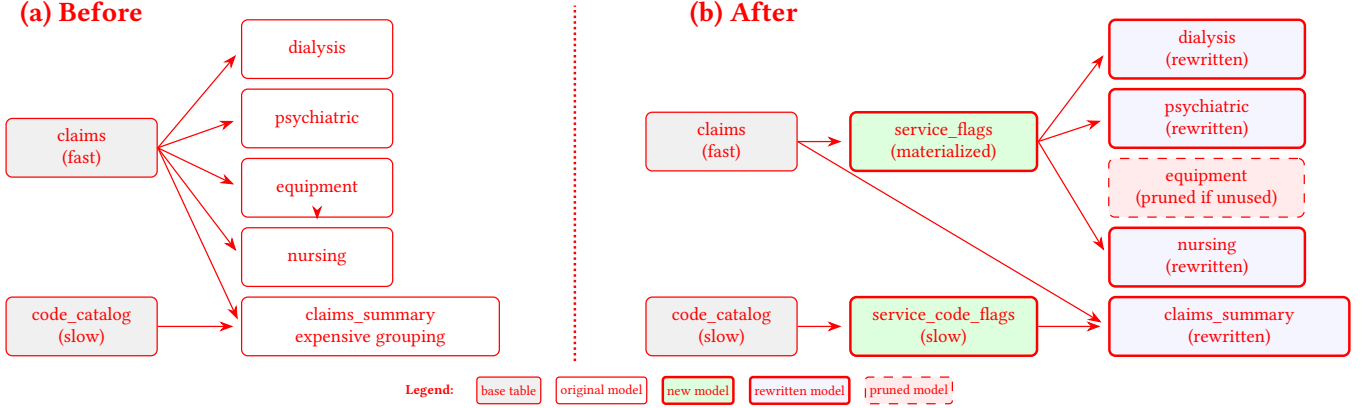


Figure 2: Healthcare SQL pipeline DAG used for three motivating examples. The dotted line separates the original DAG (a) from the cumulative rewritten DAG (b). The rewritten DAG introduces reusable service_flags, rewrites downstream models (dialysis, psychiatric, and nursing), removes the equipment dependency from nursing, and moves slow-changing reference-code grouping out of the fast claims path.

Exposes: cross-graph redundancy; reuse context Enables: non-local reuse; rewrite-materialization co-optimization Before <pre> 1 -- dialysis model 2 SELECT DISTINCT claim_id 3 FROM claims 4 WHERE bill_code LIKE '72%' 5 OR taxonomy IN (...); </pre> After <pre> 1 -- new service_flags model 2 SELECT claim_id, 3 MAX(bill_code LIKE '72%' OR taxonomy IN (...)) AS f_dia, 4 MAX(diagnosis BETWEEN '290' AND '319') AS f_psy 5 FROM claims; 6 7 -- rewritten dialysis model 8 SELECT claim_id FROM service_flags 9 WHERE f_dia; </pre> (a) Example 1: shared service flags	Exposes: edge semantics; downstream demand Enables: edge simplification; downstream-aware pruning Before <pre> 1 -- nursing model 2 SELECT * FROM claims c 3 WHERE c.facility_code IN ('31','32') 4 AND NOT EXISTS (5 SELECT 1 FROM equipment e 6 WHERE e.claim_id = c.claim_id); </pre> After <pre> 1 -- rewritten nursing model 2 SELECT * FROM claims c 3 WHERE c.facility_code IN ('31','32') 4 AND c.category <> 'equipment'; 5 6 -- equipment has no remaining consumers </pre> (b) Example 2: removing an exclusion edge	Exposes: refresh patterns; operator placement Enables: frequency-aware optimization; work placement Before <pre> 1 -- claims_summary model 2 SELECT c.id, group_code(r.description) AS grp 3 FROM claims c JOIN code_catalog r USING(code); </pre> After <pre> 1 -- new service_code_flags model 2 SELECT code, group_code(description) AS grp 3 FROM code_catalog; 4 5 -- rewritten claims_summary model 6 SELECT c.id, f.grp 7 FROM claims c JOIN service_code_flags f USING(code); </pre> (c) Example 3: precomputing slow-side logic
--	---	---

Figure 3: SQL fragments for the healthcare examples in Figure 2. Each panel corresponds to one motivating example: (a) Example 1 factors repeated service-category logic into shared flags, (b) Example 2 removes an exclusion-list dependency, and (c) Example 3 moves slow-side grouping out of the fast claims path. Each panel separates the original SQL from the rewritten SQL.

lines whose category is equipment. Figure 3b shows that the original nursing SQL anti-joins against equipment, while the rewrite uses a direct predicate over claims. This avoids building or scanning equipment as an exclusion list and removes the anti-join from nursing; we called this dependency-edge simplification in §1.

The graph also exposes downstream demand: after this rewrite, if no remaining edge leaves equipment, no final output can observe it and the model can be skipped entirely. We referred to this as downstream-aware pruning in §1.

Example 3: Refresh + placement. The claims_summary model in Figure 2(a) combines fast-changing claims with slow-changing reference codes and then applies code-grouping logic after the join. As Figure 3c shows, group_code depends only on the slow-changing code_catalog, but the original claims_summary evaluates it after joining into the high-volume claims stream. The recurring DAG exposes source update rates and consumer refresh schedules,

so moving group_code into service_code_flags lets the grouped lookup be recomputed only when the lookup table changes or consumers need it, while also applying the grouping before the join expands data volume. These are called frequency-aware optimization and pipeline-aware work placement in §1.

2.3 Why Existing Techniques Fall Short

The examples above point to a broader limitation: pipeline rewriting needs information that is distributed across the dependency graph, not only inside the query being rewritten. Single-query rewrite systems [25, 30] operate within the boundary of one query and cannot see the upstream queries that define its inputs or the downstream queries that consume its output. Multi-query optimization and common subexpression elimination [23] share intermediate computations across queries but rely on syntactic matching, so they miss the cross-query semantic equivalence illustrated in Figure 3a. Materialized view selection [1] can choose which existing results

to persist, but it does not rewrite the DAG to move slow-changing work out of a fast-refresh path, as illustrated in Figure 3c.

The gap is not that these techniques are poorly designed; it is that none treats an explicit SQL dependency graph as the unit of rewriting. Dependency-aware rewriting must combine capabilities that are usually considered separately: use producer and consumer context, find semantic reuse beyond syntactic overlap, prune work that no downstream output consumes, move expensive work after data-reducing steps, evaluate rewrites together with persistence choices, and account for recurring refresh behavior. The remainder of this paper presents an approach that integrates these capabilities into a coherent rewriting framework.

3 APPROACH

DAGSmith implements the six optimization opportunities introduced in Section 1 through three interacting implementation dimensions: graph refactoring, materialization tuning, and refresh-frequency tuning. Graph refactoring realizes dependency-edge simplification, non-local semantic reuse, downstream-aware pruning, and pipeline-aware work placement; materialization tuning realizes rewrite-materialization co-optimization by scoring each rewrite under its best table-or-view choices; and refresh-frequency tuning realizes frequency-aware optimization by aligning execution with both input-change rates and output-consumption rates. Global selection is a separate orchestration step that chooses a compatible subset of the candidates produced by these dimensions. Because our implementation and evaluation use dbt projects, this section uses *model* to denote a SQL transformation node. These dimensions are coupled: the best choice along one depends on the others, so they should not be optimized independently. We quantify these interactions with an ablation study in Section 4.3.

DAGSmith realizes these dimensions in the pipeline of Figure 4. Cheap structural analyses first scan the whole DAG and rank model groups by a fingerprint-based score that estimates the benefit of refactoring them jointly (Section 3.1). The top-ranked groups are passed to an LLM, which reasons over each region and proposes one or more concrete refactorings (Section 3.2). The structural analyses decide where in the DAG to refactor; the LLM decides what refactoring to perform. Every refactoring is adopted only after it passes **data-backed equivalence checks** run against the project’s real data: a rewrite is rejected whenever its output diverges from the original’s, so acceptance rests on observed agreement rather than the LLM’s own assurance. A refactoring’s benefit depends on how its nodes are materialized, so DAGSmith tunes each candidate’s materialization with a learned cost model and selects a compatible subset to adopt (Section 3.3). The frequency-aware optimization affects all these stages (Section 3.4): it reweights the cost-driven decisions, removes refreshes that recompute unchanged data, and contributes structural splitting candidates **using a frequency profile that combines source update rates with downstream output demand**.

3.1 Candidate Identification

Candidate identification finds where in the DAG **the four graph-refactoring optimizations from Section 1** are worth pursuing. DAGSmith confines the refactoring to small, bounded regions rather

than the whole project to ensure that the reasoning complexity over these interdependent models is tractable. Since many regions may not be worth refactoring, DAGSmith first scans the whole DAG with cheap structural analyses that rank regions by likely benefit, reserving the expensive refactoring step (Section 3.2) for the few that pay off.

The analysis must decide when two models perform the same expensive work, and the natural ways of testing for this fall short. For example, matching on SQL text fails when the same logic is written differently. Matching on plan structure goes further—multi-query optimization and view-matching are already invariant to join order, predicate order, and aliasing—but it reasons about one compiled query at a time and treats a referenced model as an opaque input. Therefore, it cannot recognize work shared across model boundaries. Figure 7 shows the problem: Models A and B compute the same join, but Model A filters its input inline while Model B reads the already-filtered input from a sibling model it references. On the compiled SQL, a plan-subtree matcher therefore sees two unrelated trees over different tables.

To address such challenges, DAGSmith matches on the operation each model computes through dataflow analysis, resolving references so that a model boundary no longer hides shared work, and reduces the two to a single reuse candidate, which the LLM and a later **data-backed equivalence check** confirm is safe to share. It further performs two analyses: reuse finds repeated expensive work to share (**non-local semantic reuse**), and prune finds work that no downstream model consumes or that runs too early (**dependency-edge simplification, downstream-aware pruning, and pipeline-aware work placement**). Only the highest-scoring regions are passed to the LLM.

Dataflow representation. Both analyses run on a *dataflow representation* rather than on SQL text: DAGSmith compiles each model into a tree of typed operators and resolves every column and `ref()` back through the models that produced it. Figure 5 shows why this is needed. Model B reads the sibling R_p , so at the SQL level its input is an opaque table name; resolving the reference substitutes R_p ’s definition, the filtered R , and reconstructs the join B actually computes over R and S . Model A reaches the same operation by filtering R inline. The two resolved trees coincide even though the `ref()` boundary hides the correspondence in the source, so the analyses can compare models by what they compute rather than by how the SQL is split across the DAG.

Reuse analysis. Reuse analysis summarizes each model’s dataflow representation by a *signature*: its expensive operations (joins and group-by aggregations), the keys they operate on, and their resolved inputs. Models whose signatures match are grouped into one *reuse candidate*; the candidates are scored and ranked by the heuristic described below.

Prune analysis. Prune analysis targets joins in two forms. A *redundant* join produces a result no downstream model consumes, confirmed by tracing column lineage from the leaf models, and can be removed outright or, when it is a semi-join whose filter is already derivable from the probed relation, collapsed to a predicate. A *mistimed* join is correct but computed too early, inflating an expensive downstream operation whose result does not depend on it; pushing it past that operation avoids carrying its product through the work.

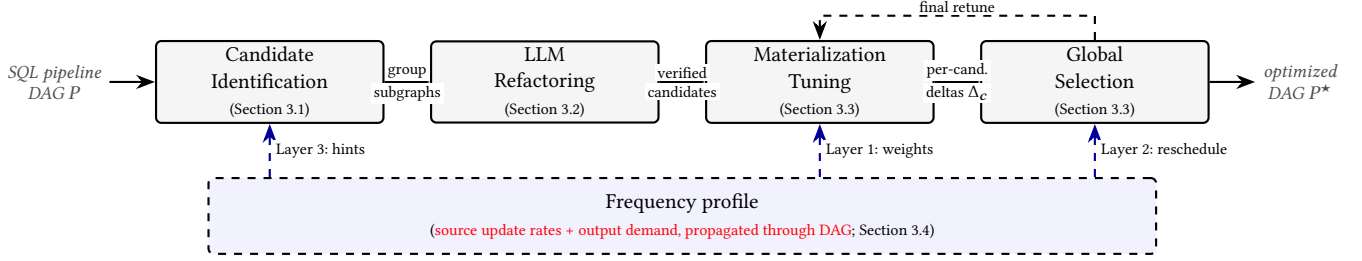


Figure 4: End-to-end pipeline of DAGSmith. The top row shows the four main stages and the artifacts that flow between them; after global selection, a final materialization pass returns the combined DAG. The dashed band underneath is the frequency dimension (Section 3.4): a single frequency profile feeds three of the four stages, contributing structural splitting hints, cost weights for materialization tuning, and schedule corrections at selection.

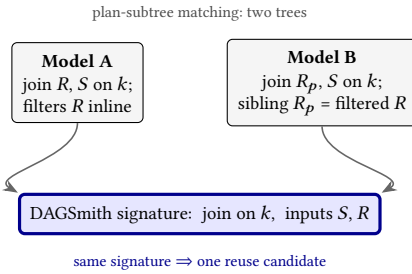


Figure 5: Candidate identification discovers shared computation across models based on dataflow, not plan shape. Models A and B compute the same join on k , but Model A filters R inline while Model B reads the already-filtered R from a sibling model R_p ; on the compiled SQL the two are different trees over different tables, while DAGSmith resolves the reference and reduces both to one signature and a single reuse candidate.

Both targets are different from local dead-code elimination in scope: whether a join’s result is used, or used too soon, is a fact about the rest of the pipeline that a single-query rewriter cannot see. Each such join becomes a *prune candidate*, scored and ranked by the same heuristic as reuse candidates.

From candidates to subgraphs. Both analyses emit candidate model groups, ranked by a single score that estimates how much work a candidate would save. The score is the product of two factors: the cost of the candidate’s expensive operation, approximated by the number of joins and group-bys it involves, and the number of models that benefit from it, namely the models that would share the computation for a reuse candidate, or the downstream models relieved of it for a prune candidate. DAGSmith processes the groups from the highest score down, so the expensive LLM stage is reserved for the regions most likely to pay off. For each group, it extracts a subgraph of the candidate models and their immediate (one-hop) upstream parents, leaving out downstream consumers and siblings; the one-hop parents supply the inputs the next stage reasons over, while each candidate’s output contract to consumers outside the subgraph stays fixed (Section 3.2). Subgraphs that exceed a node-count cap are skipped. The next subsection describes how the LLM turns these subgraphs into concrete refactorings.

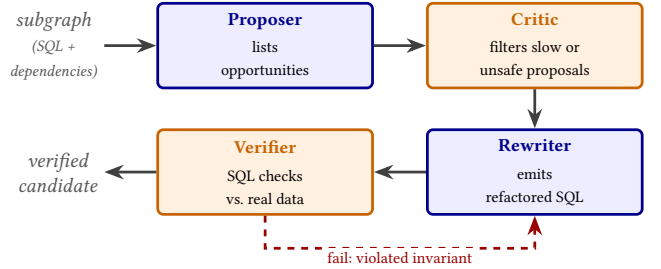


Figure 6: DAGSmith’s agentic refactoring loop. Four agents each fill one role: a *proposer* and *rewriter* (blue) generate, while a *critic* and *verifier* (amber) scrutinize. A failed equivalence check returns the violated invariant to the rewriter, which retries or reverts.

3.2 LLM Refactoring

This stage produces the refactorings that realize **the four graph-refactoring optimizations from Section 1**. As motivated in the overview, producing these cross-boundary, semantic rewrites is a challenging reasoning task, which is why DAGSmith uses state-of-the-art frontier LLMs rather than a fixed rule set. But asking an LLM to rewrite SQL that will run against a production warehouse raises three problems: the rewrite may be *slower* than the original, may be *incorrect*, or may be *too expensive to generate* when a region contains many models.

DAGSmith addresses all three with an agentic framework of four agents, each filling a single role (Figure 6): a *proposer* reads the region and lists candidate optimizations, an adversarial *critic* rejects proposals that would slow the pipeline or silently change its results, a *rewriter* turns the survivors into concrete SQL, and a *verifier* tests each rewrite against the warehouse’s real data before it is trusted. Separating proposal from criticism keeps a single prompt from having to be both creative and skeptical, and grounding acceptance in real data rather than in the model’s own assurance is what makes adopting LLM-written SQL safe.

Proposal. The first call ingests the subgraph’s SQL and dependency edges and produces a domain-level reading of the region: a short semantic summary of each model and the end-to-end business question the subgraph answers. This reading is what lets later stages recognize equivalent computations written in dissimilar SQL. The call then outputs a list of *opportunities*, each describing the work it

would remove or shrink and how—proposals to be rewritten and checked by later agents.

Filtering. A second call acts as an adversarial critic, going through the proposals and trying to break them. We keep this separate from proposal on purpose: a single prompt asked to both propose and doubt does neither well, and in practice the proposing voice often wins out. The critic looks for two failure modes. The first is a slowdown—a proposal that looks like reuse-driven savings but adds a write and a scan without cutting downstream work. The second is a correctness trap, of which Figure 7 shows one example: the proposal drops a join by reading a tag from a sibling deduplicated to one row per item, silently discarding the secondary tags the original join admitted. The critic returns one of three verdicts—*approve*, *reject*, or *conditional*—where a conditional verdict records a data property the rewrite must satisfy, carried into the next stage.

Original: join admits all tags	Proposal: drop join, read tag
<pre> 1 SELECT i.item_id, 2 MAX(CASE WHEN t.tag='A' 3 THEN 1 ELSE 0 END) AS flag_a 4 FROM ((ref('items'))) i 5 JOIN ((ref('item_tags'))) t 6 USING (item_id) 7 GROUP BY i.item_id </pre>	<pre> 1 SELECT s.item_id, 2 CASE WHEN s.tag='A' 3 THEN 1 ELSE 0 END AS flag_a 4 FROM ((ref('items_staged'))) s 5 -- items_staged keeps one tag/item; 6 -- secondary tags are silently lost </pre>

Figure 7: Correctness trap rejected by the proposal critic. The original aggregates over all tags per item; the proposal reads from a sibling that has been deduplicated to one tag per item, and silently drops the rest.

Rewriting. The opportunities that survive filtering, together with the analysis and the SQL of the affected nodes, go to a third call that produces concrete refactored SQL: it may rewrite nodes, mark nodes as removed, or introduce new shared nodes. Conditional opportunities carry their recorded constraints into this prompt. A node with consumers outside the subgraph may be rewritten internally but must produce identical output rows, since its contract with those consumers is fixed.

Equivalence verification. Filtering judges proposals one at a time, but the rewriter often bundles several into a single change—and a trap that the critic would have rejected on its own can slip back in once it is buried inside a larger rewrite. The grain-collapse shortcut of Figure 7, for instance, can resurface inside a consolidated aggregation the critic approved on other grounds. Because a bundled rewrite can be wrong even after filtering, DAGSmith checks each one against real data. A fourth call, the *correctness critic*, writes a check for every adopted opportunity: a SQL query that returns rows only when the rewrite has changed the data. These run against the target warehouse under a per-query byte cap, so any row that comes back is a real counterexample. When a check fails, the violated invariant goes back to the rewriter, which must either find a different strategy that respects it or revert to the original. The loop stops once every check passes, the LLM has no new strategy to try, or the per-group attempt budget runs out—in which case the group keeps its original SQL and contributes nothing. Because these checks run on the project’s own data, they reject any divergence that data exposes but do not amount to a formal equivalence proof, so DAGSmith does not claim guaranteed correctness. The rewrites

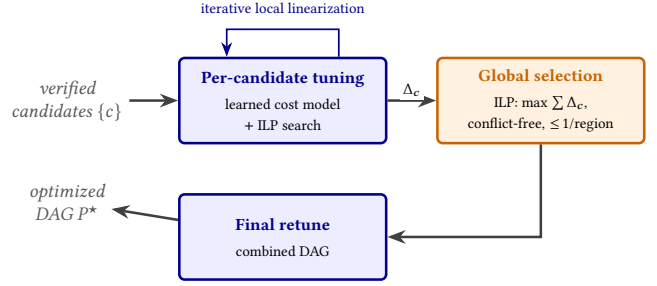


Figure 8: DAGSmith’s materialization tuning and global selection.

it adopts are meant to be confirmed by a final human inspection before deployment, a step the loop keeps lightweight by forwarding only rewrites that already survive testing on the project’s real data.

Model families and propagation. A subgraph often contains a *family* of near-duplicate models that apply the same operation to different data subsets, and running the full four-call sequence on each would be wasteful. DAGSmith instead runs it once on a representative and propagates the accepted rewrite to the rest with a lighter per-member call. Where a project has no family structure, the sequence runs over every model unchanged.

3.3 Materialization Tuning

This stage realizes **rewrite-materialization co-optimization**. A refactoring’s benefit cannot be determined in isolation: whether it helps depends on which of its nodes are stored as tables and which are inlined as views, and the best choice differs from one candidate to the next, so a candidate must be scored together with its own best materialization. **This is especially important when a rewrite creates additional reusable output: the richer rewrite may be locally more expensive, but it can become beneficial when the output is materialized once and reused by enough downstream consumers.** Even then, a second challenge remains: candidates overlap and cannot all be adopted at once, so they must be composed into a compatible set. This subsection addresses both (Figure 8): tuning one candidate’s materialization, then **separately** selecting a compatible set.

Tuning one candidate. To score a candidate, DAGSmith must find its best materialization: the assignment of each node to a table or a view that makes the whole pipeline cheapest to run, where *cost* is the total slot time of executing the entire pipeline once. That lowest cost becomes the candidate’s score.

Finding that assignment is hard because the table-or-view choice is not node-local. A table is a barrier: it is built once, and its consumers read its stored rows. A view is not: reading a view re-executes its SQL, expanding recursively through any view-parent until it reaches a table or source. Flipping a node from a table to a view therefore drops its own build cost but pushes its work, together with the view chain above it, into every consumer that reads it, once per consumer execution. A node’s cost thus depends on how its ancestors are materialized, so the choices are coupled and total cost is not a sum of per-node costs.

This coupling creates two challenges that DAGSmith handles in turn. The first is evaluation: scoring an assignment typically involves executing the pipeline, so DAGSmith instead uses a learned

cost model to enable efficient approximation. Two robust (Huber [15]) regressors, fit on instrumented runs of the original pipeline, predict each node’s output cardinality and its slot time from structural features parsed off its compiled SQL, including counts of joins and their keys, predicates by kind, group-bys, distinct operations, unions, and scans, with window, aggregation, and order-by features added for the slot-time predictor, each conditioned on the predicted cardinalities of the node’s parents. Because a view’s work folds into whoever reads it, DAGSmith accumulates each node’s features up its view chain. A node’s predicted cost then reflects the work it actually performs under the current table-or-view assignment, not just its own SQL. New nodes a refactoring introduces have no measured runs of their own and are predicted the same way, from their SQL features and their parents’ predicted cardinalities. The model only has to rank assignments within one project rather than predict absolute runtimes, so directional accuracy is enough, and the Huber loss keeps a handful of very large queries from dominating the fit.

The second is search: the best assignment must be found among exponentially many (2^n for a candidate of n nodes) possible solutions. DAGSmith navigates this with **iterative local linearization** [11, 20], a successive-approximation scheme that replaces the true nonlinear cost with a local linear surrogate, solves it, and re-linearizes at the result. Concretely, at iteration k , holding the current assignment $t^{(k)}$ fixed, DAGSmith uses the cost model to measure two local quantities: a baseline build cost $B_j^{(k)}$ for each node j (its direct parents forced to tables) and an edge-local delta $U_{ij}^{(k)}$, the extra cost at j when only parent i is inlined as a view. Treating these as constants for the iteration, it solves a small ILP for $t^{(k+1)}$:

$$\min_{t,y} \sum_{j \in V} t_j B_j^{(k)} + \sum_{i \rightarrow j \in E} y_{ij} U_{ij}^{(k)}, \quad \text{s.t.} \quad y_{ij} = (1 - t_i) \wedge t_j,$$

where y_{ij} is 1 exactly when node j is built and reads a view-parent i , and a trust-region cap on flips keeps each step where the coefficients hold. Because flipping a node changes the cardinalities its descendants see, B and U are recomputed after each move and the step repeated until the assignment stops changing; an effect the current coefficients miss, such as the cost of also flipping a sibling, then surfaces in the next round. This is what makes the coupled, nonlinear assignment tractable: instead of enumerating the 2^n assignments, each round solves one small ILP that weighs all nodes’ table-or-view choices jointly under the current cost estimates, capturing interactions that a greedy, one-node-at-a-time search would miss. The search reaches a local optimum, not a global one, but this is safe: the original DAG is tuned by the same procedure and kept as a fallback, so an under-tuned candidate simply loses to it, costing at most a missed improvement rather than a regression below the original.

Selecting a compatible set. Tuning leaves each candidate with a single number, its *delta*: how much cheaper the pipeline runs if that one refactoring is applied, with the original and the refactored version each at its own best materialization. Selection then decides which candidates to adopt. It would take all of them if it could, but two things get in the way. First, refactorings can conflict: two that rewrite, create, or remove the same model cannot both be applied. Second, the LLM sometimes proposes several alternative

refactorings for the same region, and at most one of them may be chosen. DAGSmith therefore looks for the set of candidates with the largest total delta that has no conflicts and at most one variant per region, a small problem it hands to an off-the-shelf integer-program solver. One correction remains: because each delta was measured with the candidate tuned on its own, the best table-or-view choices for the chosen combination need not match any single candidate’s, so DAGSmith retunes the combined project one final time, rerunning the same cost-model-guided search used for each candidate, now over the merged DAG with all selected refactorings in place.

3.4 Frequency-Aware Optimization

This stage realizes **frequency-aware optimization**. The stages so far treat each node’s slot time as a fixed per-execution cost. In a recurring pipeline that is suboptimal: a node’s real overhead is its per-run cost multiplied by how often it must actually run. Treating cost as per-execution therefore overvalues savings on rarely-run models and undervalues them on frequently-run ones. DAGSmith adds a frequency dimension to correct this. Effective frequencies are derived from the declared schedule at sink nodes and the observed update rates at source tables, propagated through the DAG, and applied at three layers; **the profile captures both how often inputs change and how often downstream outputs are consumed**. The first two layers leave the DAG structure intact, while the third feeds a new class of candidates back to candidate identification (Section 3.1).

Layer 1: Frequency as cost weight. Materialization tuning and selection (Section 3.3) minimize cost as if every model ran at the same rate. DAGSmith reweights their cost-model inputs by each node’s effective frequency before the decisions are made; nothing else about them changes.

Layer 2: Reconciling scheduled and effective frequency. A model’s effective frequency is the rate at which it produces *new* output, the slower of how often its consumers need fresh output (f_j^{demand}) and how often its inputs actually change (f_j^{data}):

$$f_j = \min(f_j^{\text{demand}}, f_j^{\text{data}}).$$

Where the schedule fires faster than f_j , the model re-executes and reproduces the output it produced last time. DAGSmith flags such over-scheduled models and reduces their schedule to f_j . This changes neither SQL nor materialization, only the schedule, yet every redundant execution it removes is eliminated outright.

Layer 3: Frequency-driven splitting hints. The first two layers leave the DAG structure intact; the third changes it. When a model has parents that change at different rates, it inherits the fastest, so any work inside it that depends only on slow-changing parents is recomputed at the fast rate even though its inputs have not changed. DAGSmith identifies such models and emits the location to candidate identification as a hint, alongside the reuse and prune signals. The LLM then proposes a frequency split of the kind shown in Figure 3c: the slow-side work is extracted into its own slow-cadence model and the original becomes a lightweight join, after which the candidate flows through the same filtering, rewriting, verification, and selection as any other.

4 EVALUATION

We evaluate DAGSmith on real dbt projects, organized around three questions:

- (1) How does DAGSmith compare to existing baselines for SQL pipeline optimization, and what classes of optimization account for the gap?
- (2) How much does each component of DAGSmith’s architecture contribute to the overall gain? We ablate the three **implementation dimensions** (refactoring, materialization, and frequency) and the dataflow-driven targeting that feeds the LLM stage.
- (3) Is the underlying machinery sound and affordable? We measure the effectiveness of the equivalence-verification loop at catching unsafe rewrites, the LLM cost of running the system end-to-end, and the accuracy of the cost model that drives materialization decisions.

Summary of results. On the open-source Tuva and Stripe dbt projects, DAGSmith reduces BigQuery slot-time, the unit billed under capacity pricing, by 51.1% and 47.4% to build the pipeline once, and by 67.7% and 54.8% in recurring cost averaged across three representative refresh schedules per project. Both figures beat the strongest baseline by a wide margin (on Tuva, 67.7% against its 37.9% recurring-cost reduction), because DAGSmith reaches cross-model refactoring, materialization revision, and frequency-aware rescheduling that single-query and syntactic cross-model rewriters cannot. An ablation confirms the three **implementation dimensions** are complementary, and that dataflow-guided targeting spends a fixed LLM budget more effectively than random subgraph selection (51.1% vs. 43.1% single-run). The supporting machinery is sound and affordable (Section 4.4): the verification loop reliably rejects unsafe rewrites, the LLM cost is paid once and amortized over many runs, and the learned cost model is accurate enough for the decisions it drives.

4.1 Experimental Setup

Workloads. We evaluate on two real dbt projects. *Tuva* is an open-source dbt package for healthcare data transformation. The full project comprises 842 SQL models organized across 17 modules. We use the claims preprocessing module, which contains 255 models (about 30% of the project) and is the subset whose models exhibit substantial computational complexity. We run it on a synthetic patient population of 10K, with the largest source table at 505 MB. *Stripe* is a payments-analytics project of 65 models, already factored by hand into intermediate stages so that the easy syntactic wins have been taken. We run it with the largest source table at 199 MB. The two projects together cover the range we want to evaluate: a large project that has accumulated cross-model patterns from multiple analysts, and a smaller, already-factored project where remaining gains must come from materialization, frequency, or semantic restructuring beyond syntactic deduplication.

Testbed. Experiments run on BigQuery with a 100-slot reservation on the Standard edition under capacity-based pricing. DAGSmith is not specific to BigQuery: it optimizes SQL pipelines in general, and we use BigQuery because our open-source workloads are built on them. We report slot-time as the primary cost metric, since it

Table 2: Frequency profile averages per project. Each row is the fraction of root and leaf models at each cadence, averaged across three profiles.

Project	Roots			Leaves		
	wk	day	hr	wk	day	hr
Tuva	0.18	0.57	0.25	0.07	0.20	0.73
Stripe	0.13	0.60	0.27	0.05	0.52	0.43

is the billing unit under capacity pricing, and elapsed time as a secondary latency metric. Each reported number is the median of three runs. Every LLM call in DAGSmith uses OpenAI’s GPT-5.4. To simulate realistic production schedules, we use three frequency profiles per project that assign root and leaf models to hourly, daily, or weekly cadences, with interior models derived by propagation. Within the root and leaf sets, cadences are assigned at random, with leaves skewed faster than roots, mirroring production schedules, which are driven by consumer demand at the sinks rather than the true update rate at the sources. This mismatch is exactly what the frequency dimension exploits. Table 2 summarizes the average cadence mix per project; all frequency-weighted results in this section are averaged across the three profiles.

Cost metrics. We report two views of cost. *Single-run cost* is the cost of executing the full DAG once. *Frequency-weighted cost* weights each model’s per-execution cost by the rate at which it actually needs to run, summed over the DAG; under capacity pricing this corresponds to the project’s billed cost over a period. Refactoring and materialization affect both metrics; frequency optimization affects only the frequency-weighted metric. Improvements are reported as percent reduction relative to the original project.

Baselines. We compare against three baselines spanning the design space of SQL pipeline optimization.

LearnedRewrite [32] is a rule-based per-query rewriter. It applies a portfolio of equivalence-preserving transformations to each model’s compiled SQL independently, guided by a learned policy over which rules to apply and in what order. It is competitive on standalone analytical queries but has no view of the surrounding DAG.

GenRewrite [17] is an LLM-based per-query rewriter. It introduces natural-language rewrite rules that transfer knowledge across queries and a counterexample-guided loop that iteratively corrects syntactic and semantic errors in the rewrites. Like LearnedRewrite, it operates one model at a time.

CSE is a cross-model common subexpression baseline [24], which detects subexpressions shared across a SCOPE script’s stages and materializes each once, re-optimized at the query optimizer’s memo. The memo step is not portable, since managed warehouses do not expose their optimizer’s internal state to external tools. We adapt the approach by detecting shared subexpressions at the AST level and selecting which to materialize heuristically. It is the only baseline in our comparison with cross-model scope, and represents the best a purely syntactic method can achieve on a managed warehouse without access to optimizer internals.

Table 3: Improvement over the original project (% reduction; higher is better). Medians of three runs on a 100-slot Big-Query reservation.

	Single-run		Freq-weighted	
	Elapsed	Slot	Elapsed	Slot
<i>Tuva (255 models)</i>				
LearnedRewrite	-19.6	-2.5	-20.2	-3.0
GenRewrite	9.7	17.7	21.5	15.1
CSE	12.2	34.6	27.7	37.9
DAGSmith	30.9	51.1	42.6	67.7
<i>Stripe (65 models)</i>				
LearnedRewrite	2.6	2.7	-0.5	4.5
GenRewrite	7.6	7.1	1.1	9.3
CSE	-3.2	-5.6	-3.1	-0.6
DAGSmith	40.0	47.4	36.1	54.8

4.2 Comparison with Baselines

Table 3 reports each method’s improvement over the original project on Tuva and Stripe; DAGSmith leads on every metric on both. The gap between the single-run and frequency-weighted columns is the contribution of frequency optimization, which affects only the frequency-weighted view.

Each baseline falls short for a different structural reason. We name the limit in each case, then summarize the classes of optimization that account for the gap above the strongest baseline.

LearnedRewrite: no net improvement on either project. LearnedRewrite applies equivalence-preserving rules independently to each model and therefore cannot reach any of the cross-model optimizations **identified in Section 1**, such as non-local semantic reuse and downstream-aware pruning, whose benefit depends on neighboring models. On Tuva, the one rule that fires at scale replaces a post-join `SELECT DISTINCT` with a `GROUP BY` that pre-deduplicates each join input. When the join key is near-unique, this grouping is nearly free and lets the query skip an expensive `DISTINCT` over the wide joined result, making the rewrite faster. When many rows share each key value, the warehouse already runs the original efficiently as a semi-join, and the extra grouping is wasted work that slows the rewrite down. Across the models the rule touches, wins and losses are roughly balanced. On Stripe, which is already hand-factored, LearnedRewrite’s rules find little to match and it changes almost nothing. Overall, LearnedRewrite barely changes either project’s cost: rule-based per-query rewriting has little left to do once the warehouse planner has already handled the simple per-query cases, and the changes that would actually help these projects span multiple models, which it cannot do.

GenRewrite: cuts cost within a single model, not across the pipeline. GenRewrite restructures a single model to compute the same result more cheaply, such as replacing an $O(n^2)$ self-join with $O(n)$ window functions, a rewrite current rule-based systems do not support; on Tuva these account for its 17.7% slot-time reduction. The gain is bounded because every rewrite stays inside a single model: GenRewrite cannot share work across models, change materialization, or skip refreshes. It can also regress, because it rewrites each model on its own. The same expensive self-join appears in

several near-identical models. For one, GenRewrite removed it with the $O(n)$ window-function rewrite above; for a near-identical sibling, it instead produced three self-joins that cost more than twice as much as the original. The same good rewrite would have worked for the sibling, but GenRewrite cannot tell that the two are the same and reuse it. On Stripe, already hand-factored, fewer such rewrites are available and the gain is smaller (a 7.1% slot-time reduction). GenRewrite optimizes each model without seeing the others, which both caps its savings and lets it regress sometimes.

CSE: syntactic matching sees duplication, not equivalence.

CSE matches subexpressions across models by their SQL structure, and is the only baseline that works across models. Where the same SQL recurs, it does well: Tuva has a large family of models that each scan and classify the same claims source with nearly identical SQL, and CSE detects the repeated work and runs it once for the whole family, which is most of its 34.6% Tuva slot-time reduction. Where the SQL differs, it cannot recognize the reuse: six models compute the same per-encounter totals, each from different inputs and in different SQL, and CSE’s structural matching does not see them as the same, recovering little of the shared work. On Stripe, CSE slightly increases cost rather than reducing it, the only baseline to do so. Extracting a subexpression into a shared model only pays off when enough consumers reuse it; on Stripe, what CSE extracts is reused by too few consumers to cover the cost of building and scanning it.

DAGSmith: four patterns that the baselines structurally cannot reach. DAGSmith’s advantage over the strongest baseline comes from four recurring patterns. The first two are cross-model refactorings that go beyond matching SQL text; the last two are materialization choices that no SQL rewriter can change. We give one example of each from the workloads.

(1) *Sharing equivalent work that is written in different SQL.* Models that compute the same thing over different inputs are folded into one shared model, even when their SQL does not match. Take the six encounter models from above, which compute the same per-encounter totals but cannot be matched on SQL text: DAGSmith builds one shared rollout that carries the encounter type as a column and replaces all six with short lookups, cutting their slot-time by about 77%, against CSE’s 14%. CSE detects that the six share structure but skips them, because their differing encounter-type literals leave the SQL not byte-identical; the equivalence is clear only once the LLM reads what each model computes.

(2) *Replacing repeated wide-table joins with a narrow shared intermediate.* In one Tuva consumer, a union of 32 branches, seven of them each join a different encounter model to the same wide claim-line staging table only to read two of its columns. DAGSmith materializes those two columns once as a narrow intermediate and points all seven joins at it, cutting the consumer’s slot-time by about 74%. The saving is not from scanning less data: the warehouse already prunes the unused columns, and the rewrite even processes slightly more bytes. It comes from the joins, since probing a two-column intermediate builds a far smaller hash table than probing the wide staging table. CSE cannot find this, because each branch joins a different upstream model and no two branches are the same SQL fragment to match.

(3) *Storing a staging chain as views so the planner can optimize across it.* A staging model stored as a table is built in full and read back as a fixed result the planner cannot optimize across. Stored as a view, it is just SQL the planner folds into the query that reads it, letting it push that query’s filters down to the staging’s sources and drop intermediate work the query never uses. DAGSmith treats this table-or-view choice as something to tune. On Stripe, switching the fifteen staging models behind a wide reporting model from tables to views, with no change to any SQL, cut the data it processes by about a third and its slot-time by about 60%. Materializing each staging as a table seems like it should help by computing it once, but a table is built in full before the reporting query’s filters can apply, leaving the planner nothing to trim; as views, those filters reach the sources, and the whole query is optimized at once. The table-or-view choice is a configuration setting, not part of the SQL; a tool that only rewrites compiled SQL never sees it.

(4) *Removing intermediates that feed only one model.* On Tuva, six normalize-and-vote stages were each materialized as a table, even though each fed only a single downstream model. Materializing an intermediate pays off only when several models reuse it; for a single consumer, the build cost is wasted. DAGSmith’s tuner flips these single-consumer stages to views, folding each into its one consumer and dropping the family’s slot-time by about 24%. CSE keeps all six as tables and gets worse on the same family, because it cannot change materialization at all. No SQL rewriter can do this, because the choice is a materialization setting, not part of the SQL.

Frequency optimization accounts for the gap between the single-run and frequency-weighted columns of Table 3, realized through the three layers of Section 3.4. The first two carry most of it: reweighting each model’s cost by how often it actually runs steers tuning toward the models that dominate the recurring bill, and lowering the schedule of models that fire faster than their inputs change removes redundant reruns outright. The third layer, splitting slow-changing work out of a fast-refreshing model, is the most structural change and adds a modest 3.6% reduction, because splittable models are rarer than they appear. Splitting needs both parents that change at different rates and a slow side separable from the fast path, and most models fail one of the two: all their inputs change at the same rate, or the slow and fast inputs are entangled at the row level. It pays off only when the slow side merely appends columns, as in a left join that adds a lookup description. Like materialization, none of this is visible to a tool that sees only the compiled SQL.

The two projects lean on these patterns differently: DAGSmith’s advantage on Tuva comes mostly from patterns 1 and 2 (cross-model refactoring), and on Stripe from pattern 3 (materialization). The same pipeline produces both because it applies whatever fits each project rather than relying on a single mechanism. The next subsection quantifies each component’s contribution.

4.3 Component Ablation

Section 4.2 described the *classes of optimization* responsible for DAGSmith’s gap above the baselines. This subsection asks how much each of DAGSmith’s own architectural components contributes and how they interact. We ablate along two axes: the three **implementation dimensions** of LLM refactoring (R; Section 3.2), materialization tuning (M; Section 3.3), and frequency optimization (F; Section 3.4);

Table 4: Tuva: dimension ablation (% reduction over the original project, averaged across the three Tuva profiles in Table 2). R = refactoring, M = materialization tuning, F = frequency optimization. Bold marks the best variant in each column.

Variant	Elapsed	Slot
original	–	–
F only	13.2	11.0
M only	34.7	37.0
R only	40.0	49.1
M + F	31.0	54.0
R + M	43.1	54.7
R + M + F (full)	42.6	67.7

Table 5: DAGSmith’s coverage of Tuva’s 255 models. Percentages are of the 255 original models.

Category	Count
SQL refactored only	90 (35.3%)
Materialization retuned only	13 (5.1%)
Both refactored & retuned	3 (1.2%)
New models created	11 (4.3%)
Total models modified	117 (45.9%)

and the dataflow analysis that targets the LLM stage (Section 3.1). We ablate on Tuva alone, since it is the larger and more structurally varied project and exercises all three dimensions meaningfully, whereas on the smaller Stripe project, attributing gains to individual components is unreliable.

4.3.1 Three Optimization Dimensions. Table 4 reports the improvement of each subset of {R, M, F} over the original Tuva project.

The numerics admit four observations, each reflecting a property of how the dimensions are designed to interact.

Refactoring and materialization mostly remove work parallel to the critical path. Recall that slot measures the total CPU-time the project consumes, while elapsed measures wall-clock time from the first model to the last. For every variant that includes refactoring or materialization, the slot reduction exceeds the elapsed reduction (R-only, for instance, cuts slot by 49.1% but elapsed by 40.0%), because DAGSmith removes work that runs parallel to the critical path more readily than work on it: consolidating many sibling models into one shared intermediate saves slot proportional to the number of siblings but shortens elapsed only by the slowest sibling’s share. Frequency optimization is the exception, cutting elapsed and slot by comparable amounts (13.2% vs. 11.0%), because it removes whole scheduled executions rather than parallel work. The same mechanism explains the full pipeline: adding F on top of R+M raises the slot reduction sharply (54.7% to 67.7%) while elapsed is essentially unchanged (43.1% to 42.6%, a difference within run-to-run noise), because the redundant executions F eliminates lie off the critical path. F thus lowers billed cost without affecting wall-clock latency, which is exactly its intended effect.

R-only is close to R+M because R is not pure refactoring. When DAGSmith’s refactoring stage introduces a new model, it

Table 6: Dataflow Analysis: targeting ablation on Tuva (% slot reduction over the original project) under a fixed LLM-call budget across all arms. Whole-DAG refactoring produces no runnable project, so it has no cost to report (see text).

Selection rule	Single-run slot	Freq-weighted slot
Whole-DAG	no valid DAG	
Random subgraphs	43.1	58.0
Dataflow-guided (DAGSmith)	51.1	67.7

also assigns that model an optimal materialization as part of producing the candidate (Section 3.3). R-only therefore already runs the materialization tuner on every model it creates, so its gap to R+M in Table 4 does not measure the value of materialization tuning; it measures only the slice R-only leaves out, the retuning of pre-existing models. That slice is small: only 16 of Tuva’s 255 models have their materialization retuned (Table 5), and the R-only to R+M gap reflects exactly those 16. M-only trails R-only for the complementary reason: it revises materialization only on the original DAG and cannot create models. Even so, M cannot be dropped in favor of R+F. On Tuva the materialization that R assigns to its new models already sits close to the project-wide optimum, so M’s extra retuning adds little; this overlap varies across projects, and where it is smaller M contributes more.

Frequency’s increment depends on how much over-scheduled work the prior dimensions leave behind. F alone delivers 11.0%, adds 17.0% on top of M (54.0% vs. 37.0%) and 13.0% on top of R+M (67.7% vs. 54.7%). F’s reach is bounded by the over-scheduled work present in the project, and that pool depends on the prior dimensions: refactoring can move work onto the right cadence and remove it from F’s surface (splitting a fast-cadence model into a slow rollout and a fast lookup), while materialization tuning leaves the over-schedule pool largely intact, so F has more to remove on top of M than on top of R+M. F’s increment never collapses to zero, which shows it is orthogonal to R and M rather than a redundant pass; and because that increment varies with the prior dimensions, the three interact rather than add independently.

4.3.2 Dataflow Analysis. This experiment isolates the contribution of the dataflow analysis that produces DAGSmith’s candidate set (Section 3.1): does structural targeting direct the LLM stage to more productive regions of the DAG than alternatives that do not use it? We hold the LLM-call budget fixed across every configuration we compare, so the comparison reflects how well a fixed budget is spent rather than how many invocations are made. In the standard pipeline, dataflow analysis selects at most 20 subgraphs on Tuva, each capped at 60 nodes with 3 diagnostic-and-correction attempts; we hold this budget fixed across two alternatives. Table 6 reports the result.

Whole-DAG refactoring. The first alternative hands the entire project to the LLM under the same 3-attempt budget, testing whether bounding its attention to a region is needed at all. On Tuva the whole DAG fits in context (134k input tokens) and analysis succeeds, surfacing most of the opportunities our pipeline finds, but implementation never produces a runnable project. The response

must restate every rewritten model in full, so it grows long enough to truncate mid-output and leave dangling references; cross-model references grow inconsistent as a fix to one model breaks an assumption in another; and because every broad attempt breaks somewhere, the correction loop retreats in scope across the three attempts (51, then 35, then 8 models). A broad rewrite captures most of the gain but touches too many interdependent models to verify, while one narrow enough to verify captures only a fraction. Dataflow analysis resolves this by making each subgraph small enough to verify while still reaching the high-value regions one at a time.

Random subgraphs. The second alternative changes only the selection rule, holding the subgraph count, size cap, and refactoring procedure fixed: we draw a model uniformly at random, take its parents, children, and siblings as a subgraph, and repeat until the count matches the standard pipeline, discarding any neighborhood over the node cap. This isolates the contribution of *where* the pipeline chooses to look. As Table 6 shows, random selection reaches 43.1% single-run and 58.0% frequency-weighted slot reduction, against 51.1% and 67.7% for dataflow-guided selection. The gap is a structural mismatch: an edge-based subgraph groups a model with its dependency neighbors, but the models worth refactoring together are elsewhere in the project, computing the same pattern over different inputs without sharing a single edge, so no edge-based neighborhood contains both and widening it only adds irrelevant dependency-adjacent models. Dataflow-guided selection groups on the structure of the computation rather than on dependency edges, placing the models that compute the same pattern in one subgraph regardless of where they sit in the DAG.

4.4 System Characterization

This subsection asks whether the machinery behind the gains is sound and affordable: the equivalence-verification loop that catches unsafe rewrites, the LLM cost of running DAGSmith end to end, and the accuracy of the learned cost model that drives materialization.

4.4.1 Equivalence-Verification Effectiveness. The question for the verification loop (§3.2) is whether it catches unsafe rewrites without discarding safe ones, under a budget of three correction attempts per subgraph. Of the 20 candidate subgraphs on Tuva, 12 fit within the size cap and reach verification (Table 7). Only a quarter pass on the first attempt, and the rewrites the diagnostics reject fall into two recurring patterns a per-query checker cannot detect. The dominant one is a grain collapse: the rewrite reads a value from a pre-duplicated sibling that keeps one row per key, silently dropping the secondary rows the original aggregates over (Figure 7). The second is algorithm substitution: a cheaper algorithm whose output differs from the original, such as a window-based interval merge that yields different groupings than the pairwise merge it replaces. Yet none of the 12 exhausts the budget: every rejected subgraph passes within a second attempt once its violated invariants are returned to the rewriter, so the loop is a gate rather than a dead end. On Stripe most candidates pass on the first attempt, since its smaller DAG and simpler SQL leave the rewrites less error-prone.

4.4.2 LLM Cost. Running DAGSmith is a one-time, design-time cost, paid once per project and amortized over every scheduled run

Table 7: Equivalence-verification outcomes on Tuva’s 12 refactoring subgraphs (three-attempt budget each).

Outcome	Subgraphs
Pass within 1 attempt	25% (3/12)
Pass within 2 attempts	100% (12/12)
Never pass (budget exhausted)	0%

Table 8: End-to-end LLM cost of optimizing Tuva with GPT-5.4, by pipeline stage.

	Proposal	Filtering	Rewriting	Verification	Total
Cost (\$)	2.00	0.96	5.68	2.88	11.51
Share (%)	17	8	49	25	100

of the optimized pipeline. Optimizing Tuva’s 375 models with GPT-5.4 cost \$11.51 and 2.4M tokens, about \$0.03 per model, dominated by the rewriting stage (Table 8). This is small beside the recurring warehouse cost the optimized pipeline removes on every run.

4.4.3 Cost Model Accuracy. The learned cost model only needs to rank refactoring and materialization options within a project, not predict absolute runtimes or generalize across projects. Fit per project on the 156 instrumented Tuva models, its slot-time predictor, the quantity the ILP minimizes, reaches $R^2=0.74$ from static SQL and DAG features alone. Cardinality is harder ($R^2=0.28$), since row counts depend on data content that static features cannot see, but directional estimates are enough for ranking.

5 RELATED WORK

Positioning. DAGSmith is, to the best of our knowledge, the first system to treat the explicit pipeline DAG as **the unit of dependency-aware rewriting**, restructuring node boundaries and deciding refactoring, materialization, and refresh frequency jointly. We now discuss several closest lines of work.

Single-query SQL rewriting. Rule-based rewriters [30, 32], program-synthesis rewriters [10], and LLM-based rewriters [16, 17, 25] transform a query into an equivalent form before the database optimizer plans it; two of our baselines, LearnedRewrite [32] and GenRewrite [17], are of this kind. All of them optimize one statement in isolation and leave the surrounding dependency graph unchanged, so rewrites whose benefit depends on neighboring transformations, such as reusing logic across models or removing work that no downstream output consumes, fall outside their scope.

Multi-query optimization and view selection. Multi-query optimization [22, 23] and common-subexpression sharing [24, 31] reuse intermediate results across statements, and materialized-view selection [1, 14] precomputes chosen results and rewrites consumers to read them; our cross-model CSE baseline follows this line [24]. These techniques match on syntactic structure and select persistence among candidates that already exist. They do not recognize the same logical computation written in dissimilar SQL, as in Figure 3a, and they reuse the computation behind existing results without considering whether each result is needed downstream, so

they keep every output in place. DAGSmith instead reads downstream demand to trim redundant computation and simplify the dependency graph (Figure 3b).

View maintenance and refresh scheduling. Incremental view maintenance [4] and streaming materialized-view systems [19, 29] keep derived results fresh by processing only the rows that change, which lowers the cost of each refresh. DAGSmith addresses freshness from the opposite direction: rather than reducing the refresh cost of specific models, it aligns how often each node runs with how often its inputs change and its outputs are consumed, and it splits a node along a frequency boundary when slow-changing work is trapped on a fast cadence (Figure 3c). The two approaches are complementary and could compose.

Workflow orchestration. dbt [13], SQLMesh [26], Airflow [2], Dagster [5], Prefect [21], Argo [3], Luigi [18], and Databricks Workflows [6] express recurring transformations and their dependencies as a DAG, the setting DAGSmith targets. Their schedulers optimize execution order, invalidation, retries, and resource allocation, but they leave the SQL semantics of each node untouched, which is where DAGSmith’s reductions originate.

6 CONCLUSION

Explicit SQL pipeline DAGs create an optimization setting that is richer than traditional one-query-at-a-time rewriting. Once dependencies, **downstream demand**, **final outputs**, materialization choices, and refresh behavior are visible, the optimizer can restructure the pipeline itself rather than merely improve individual statements. DAGSmith is a step toward that goal: it treats dependency-aware pipeline optimization as a joint problem over graph structure, materialization, and frequency, and combines structural analysis with LLM-guided refactoring and verification to explore that space.

Disclaimer. The authors used generative AI tools to assist with editing and polishing the manuscript. All technical ideas, claims, analyses, and final text were reviewed and approved by the authors.

REFERENCES

- [1] Sanjay Agrawal, Surajit Chaudhuri, and Vivek R. Narasayya. 2000. Automated Selection of Materialized Views and Indexes in SQL Databases. In *Proceedings of the 26th International Conference on Very Large Data Bases*. 496–505.
- [2] Apache Airflow. 2026. DAGs. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>. Accessed 2026-05-26.
- [3] Argo Workflows. 2026. DAG. <https://argo-workflows.readthedocs.io/en/latest/walk-through/dag/>. Accessed 2026-05-26.
- [4] José A. Blakeley, Per-Åke Larson, and Frank Wm. Tompa. 1986. Efficiently Updating Materialized Views. In *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 61–71. <https://doi.org/10.1145/16894.16861>
- [5] Dagster Labs. 2026. Software-defined assets. <https://docs.dagster.io/>. Accessed 2026-05-26.
- [6] Databricks. 2026. Configure task dependencies. <https://docs.databricks.com/aws/en/jobs/run-if>. Accessed 2026-05-26.
- [7] Databricks. 2026. What is the medallion lakehouse architecture? <https://learn.microsoft.com/en-us/azure/databricks/lakehouse/medallion>. Accessed 2026-05-26.
- [8] dbt Labs. 2024. The next big step forwards for analytics engineering. <https://www.getdbt.com/blog/analytics-engineering-next-step-forwards>. Accessed 2026-05-26.
- [9] dbt Labs. 2025. 2025 State of Analytics Engineering Report. <https://www.getdbt.com/resources/reports/state-of-analytics-engineering-2025>. Accessed 2026-05-26.
- [10] Rui Dong, Jie Liu, Yuxuan Zhu, Barzan Mozafari, Xinyu Wang, and Cong Yan. 2023. SlabCity: Whole-Query Optimization using Program Synthesis. <https://arxiv.org/abs/2310.12345>.

- //par.nsf.gov/servlets/purl/10451715.
- [11] R. E. Griffith and R. A. Stewart. 1961. A Nonlinear Programming Technique for the Optimization of Continuous Processing Systems. *Management Science* 7, 4 (1961), 379–392. <https://doi.org/10.1287/mnsc.7.4.379>
 - [12] Tristan Handy. 2026. What, exactly, is dbt? <https://www.getdbt.com/blog/what-exactly-is-dbt>. Accessed 2026-05-30.
 - [13] Tristan Handy. 2026. What, Exactly, Is dbt? <https://www.getdbt.com/blog/what-exactly-is-dbt>. Accessed: 2026-05-30.
 - [14] Venky Harinarayan, Anand Rajaraman, and Jeffrey D. Ullman. 1996. Implementing Data Cubes Efficiently. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 205–216. <https://doi.org/10.1145/233269.233333>
 - [15] Peter J. Huber. 1964. Robust Estimation of a Location Parameter. *The Annals of Mathematical Statistics* 35, 1 (1964), 73–101. <https://doi.org/10.1214/aoms/1177703732>
 - [16] Zhaodonghui Li, Haitao Yuan, Huiming Wang, Gao Cong, and Lidong Bing. 2024. LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Efficiency. *Proc. VLDB Endow.* 18, 1 (2024), 53–65. <https://doi.org/10.14778/3696435.3696440>
 - [17] Jie Liu and Barzan Mozafari. 2026. GenRewrite: Query Rewriting via Large Language Models. *Proceedings of the ACM on Management of Data* 4, 1 (SIGMOD (2026)), 1–26.
 - [18] Luigi Contributors. 2026. Building workflows. <https://luigi.readthedocs.io/en/stable/workflows.html>. Accessed 2026-05-26.
 - [19] Frank McSherry. 2022. Materialize: A Platform for Building Scalable Event Based Systems. In *Proceedings of the 16th ACM International Conference on Distributed and Event-Based Systems*. Association for Computing Machinery, New York, NY, USA, 3. <https://doi.org/10.1145/3524860.3544408>
 - [20] F. Palacios-Gómez, L. Lasdon, and M. Engquist. 1982. Nonlinear Optimization by Successive Linear Programming. *Management Science* 28, 10 (1982), 1106–1120. <https://doi.org/10.1287/mnsc.28.10.1106>
 - [21] Prefect Technologies. 2026. Tasks. <https://docs.prefect.io/v3/concepts/tasks>. Accessed 2026-05-26.
 - [22] Prasan Roy, S. Seshadri, S. Sudarshan, and Siddhesh Bhole. 2000. Efficient and Extensible Algorithms for Multi Query Optimization. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 249–260. <https://doi.org/10.1145/342009.335419>
 - [23] Timos K. Sellis. 1988. Multiple-Query Optimization. *ACM Transactions on Database Systems* 13, 1 (1988), 23–52.
 - [24] Yasin N. Silva, Per-Åke Larson, and Jingren Zhou. 2012. Exploiting Common Subexpressions for Cloud Query Processing. In *IEEE 28th International Conference on Data Engineering (ICDE 2012)*, Washington, DC, USA (Arlington, Virginia), 1-5 April, 2012, Anastasios Kementsietsidis and Marcos Antonio Vaz Salles (Eds.). IEEE Computer Society, 1337–1348. <https://doi.org/10.1109/ICDE.2012.106>
 - [25] Zhaoyan Sun, Xuanhe Zhou, Guoliang Li, Xiang Yu, Jianhua Feng, and Yong Zhang. 2025. R-Bot: An LLM-based Query Rewrite System. *Proc. VLDB Endow.* 18, 12 (2025), 5031–5044. <https://doi.org/10.14778/3750601.3750625>
 - [26] Tobiko Data. 2026. SQLMesh Overview. <https://sqlmesh.readthedocs.io/en/stable/concepts/overview/>. Accessed 2026-05-26.
 - [27] Tuva Health. 2026. The Tuva Project. <https://thetuvaproject.com/>. Accessed: 2026-06-04.
 - [28] Adrian Vogelsang, Michael Haubenschild, Jan Finis, Alfons Kemper, Viktor Leis, Tobias Muehlbauer, Thomas Neumann, and Manuel Then. 2018. Get Real: How Benchmarks Fail to Represent the Real World. In *Proceedings of the Workshop on Testing Database Systems (DBTest '18)*. Association for Computing Machinery, New York, NY, USA, Article 4, 6 pages. <https://doi.org/10.1145/3209950.3209952>
 - [29] Yanghao Wang and Zhi Liu. 2022. A Sneak Peek at RisingWave: A Cloud-Native Streaming Database. In *Proceedings of the 16th ACM International Conference on Distributed and Event-Based Systems*. Association for Computing Machinery, New York, NY, USA, 190–193. <https://doi.org/10.1145/3524860.3543284>
 - [30] Zhaoguo Wang, Zhou Zhou, Yicun Yang, Haoran Ding, Gansen Hu, Ding Ding, Chuzhe Tang, Haibo Chen, and Jinyang Li. 2022. WeTune: Automatic Discovery and Verification of Query Rewrite Rules. In *Proceedings of the 2022 International Conference on Management of Data*. 94–107. <https://doi.org/10.1145/3514221.3526125>
 - [31] Jingren Zhou, Per-Åke Larson, Johann Christoph Freytag, and Wolfgang Lehner. 2007. Efficient exploitation of similar subexpressions for query processing. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, Beijing, China, June 12-14, 2007*, Chee Yong Chan, Beng Chin Ooi, and Aoying Zhou (Eds.). ACM, 533–544. <https://doi.org/10.1145/1247480.1247540>
 - [32] Xuanhe Zhou, Guoliang Li, Chengliang Chai, and Jianhua Feng. 2021. A Learned Query Rewrite System using Monte Carlo Tree Search. *Proc. VLDB Endow.* 15, 1 (2021), 46–58. <https://doi.org/10.14778/3485450.3485456>